Relatório Técnico: Sala de Aula em OpenGL

Computação Gráfica · José Riquelme Teixeira da Silva, Erasmo da Silva Sá Junior e Victor André Lopes Brasileiro · Nov 1, 2026

Introdução

Construímos uma sala de aula em 3D que pode ser explorada com teclado e mouse, usando OpenGL 3.3 core profile e C++17. Nosso objetivo era chegar a uma cena convincente sem recorrer a motores gráficos: toda a geometria, a iluminação e as interações foram escritas por nós.

As únicas bibliotecas usadas foram GLFW (janela e entrada), GLAD (carregamento das funções do OpenGL), GLM (matemática de vetores e matrizes) e stb_image (leitura das imagens de textura). O build é feito com CMake, e as quatro bibliotecas são compiladas junto com o projeto a partir da pasta `lib/`, o que permitiu que os três membros trabalhassem em Windows, Linux e WSL com o mesmo procedimento.

Estrutura base e pipeline de renderização

A base do projeto é um loop de renderização que, a cada quadro, lê a entrada, atualiza as animações e desenha a cena. Usamos exclusivamente o pipeline programável, com um vertex shader e um fragment shader escritos em GLSL; nenhuma função do pipeline fixo (`glBegin`, `glMatrixMode`) foi utilizada.

- **Formato de vértice:** todo vértice carrega posição, normal e coordenada UV. Definimos esse formato logo no início porque a iluminação depende da normal e a textura depende da UV; um mesh sem algum deles ficaria incompleto.
- **Mesh reutilizável:** criamos uma única estrutura de mesh que envia os vértices à GPU por meio de VAO, VBO e EBO. O EBO (índices) evita repetir vértices compartilhados entre triângulos, e todos os objetos da cena usam essa mesma estrutura.
- **Teste de profundidade:** ativamos o depth buffer para que as superfícies mais próximas da câmera escondam as que estão atrás, independentemente da ordem de desenho.
- **Organização em camadas:** separamos o código em `core/` (entrada e câmera), `gfx/` (OpenGL), `objects/` (os objetos da sala) e `scene/` (a montagem). Cada objeto é modelado no próprio espaço, com o pivô na origem, e só a cena decide onde ele fica. Isso permitiu que cada membro trabalhasse nos seus arquivos sem conflitos.
- **Modos de depuração:** o fragment shader também pode exibir a normal, a UV ou a cor sem luz de cada peça. Esses modos nos ajudaram a encontrar normais invertidas e UVs esticadas durante a modelagem.

Modelagem procedural

Todos os nove objetos da sala, além das paredes, do piso e do teto, foram modelados diretamente no código; nenhum modelo foi importado. Adotamos 1 unidade = 1 metro e o eixo Y para cima, o que nos deixou usar medidas reais de móveis escolares.

Para a maior parte das peças, escrevemos três primitivas paramétricas: caixa, cilindro e plano. Cada uma gera posições, normais e UVs, e recebe um tamanho físico de textura. Assim a UV é calculada em metros, e uma perna de mesa fina não estica a madeira como o tampo. Quando a forma não cabia nas primitivas, montamos o mesh vértice a vértice.

Objeto	Como foi construído
Ventilador de parede	Suporte, motor e grades com primitivas; pás com perfil de largura variável gerado por uma função seno. Rotor separado do corpo para poder girar
Cadeira	Pernas e travessas cilíndricas, assento e encosto (inclinado 8°) em caixas finas
Quadro	Painel, moldura, cantos e bandeja, com apagador e dois marcadores como peças separadas
Carteira	Tampo de 0,60 × 0,45 m, estrutura de tubo de 22 mm e porta-livros de arame
Luminária e interruptor	Calha com refletor em “W”, aletas e dois tubos; a tecla do interruptor bascula sem reconstruir geometria
Mesa do professor	Tampo, laterais, painel frontal e gaveteiro com três gavetas e puxadores
Projetor	Corpo, lente como peça independente e suporte de teto com haste
Tela retrátil	Caixa, lona de 2,0 × 1,5 m, barra inferior, cordinha e puxador; pivô da lona no topo
Lixeira	Tronco de cone oco gerado manualmente: paredes externa e interna, borda com espessura e dois fundos

Carteira e cadeira compartilham a mesma espessura de tubo e o mesmo material metálico, definidos num arquivo comum. Objetos repetidos, como as seis carteiras, as seis cadeiras e as quatro luminárias, reaproveitam os mesmos meshes na GPU e só mudam a transformação.

Transformações e grafo de cena

A cena é uma árvore de nós em que cada peça guarda sua transformação relativa ao pai. Ao desenhar, multiplicamos a matriz do pai pela do filho, de modo que mover a raiz de um objeto leva todas as suas peças junto. A matriz de cada peça segue a ordem T · R · S: aplicada ao vértice da direita para a esquerda, ela primeiro escala, depois gira e por último posiciona.

```
M_{\text{peça}} = M_{\text{pai}} \cdot T \cdot R \cdot S
```

As três transformações exigidas aparecem na cena:

- **Translação:** no posicionamento de todos os objetos na sala e no deslocamento da câmera.
- **Rotação:** nas hélices do ventilador, que giram em torno do centro do rotor com `T(centro) · Rz(θ)`. Como o rotor é filho do corpo, ele herda a posição do ventilador na parede, e só é preciso atualizar o ângulo θ.
- **Escala:** na lona da tela retrátil, com `T(fenda) · S(1, s, 1)`. Colocamos o pivô da lona no topo para que, ao escalar em Y, ela cresça apenas para baixo, sem se descolar da caixa.

Um cuidado importante foi com as normais. Quando uma peça é escalada de forma não uniforme, multiplicar a normal pela matriz model a deixa torta, e a luz fica errada. Por isso calculamos, para cada peça, a matriz das normais `transpose(inverse(mat3(model)))`, que mantém a normal perpendicular à superfície. Com a tela totalmente recolhida (s = 0), essa matriz não tem inversa, então nesse caso a lona simplesmente não é desenhada.

Câmera e navegação

Implementamos uma câmera livre em primeira pessoa, controlada por W/A/S/D e pelo mouse. A orientação é guardada em dois ângulos, yaw (para os lados) e pitch (para cima e para baixo), convertidos no vetor de direção por seno e cosseno. O pitch é limitado a ±89°, porque olhar exatamente para cima deixaria a direção paralela ao eixo Y e a matriz view ficaria indefinida.

- **Matriz view:** gerada com `lookAt`, a partir da posição da câmera e do ponto para onde ela olha.
- **Matriz de projeção:** perspectiva com campo de visão vertical de 60°, planos near e far em 0,1 m e 100 m e proporção atualizada quando a janela muda de tamanho.
- **Movimento por deltaTime:** a velocidade é de 2 m/s, multiplicada pelo tempo real entre quadros. Assim a navegação tem a mesma velocidade em qualquer computador. Normalizamos o vetor de movimento para que andar na diagonal não seja mais rápido.

- **Captura do cursor:** o clique esquerdo captura o mouse e o Esc o libera. Na primeira leitura após a captura, só registramos a posição, para evitar um salto brusco da câmera.

Também criamos um modo de inspeção: as teclas de 1 a 0, `[` e `]` isolam cada objeto e posicionam a câmera para enquadrá-lo. Esse recurso foi útil para revisar os modelos durante o desenvolvimento e deve ajudar na apresentação.

Iluminação e materiais

A iluminação usa o modelo de Blinn-Phong, calculado por fragmento no fragment shader. Escolhemos o cálculo por fragmento, e não por vértice, porque as peças têm poucos vértices e o brilho ficaria borrado ou sumiria entre eles. A cor final de cada ponto é a soma de quatro contribuições:

- **Emissiva:** luz própria do material, usada nos tubos das luminárias acesas e na lente do projetor ligado.
- **Ambiente:** um valor constante que impede que as faces viradas para longe das luzes fiquem totalmente pretas.
- **Luz de preenchimento:** uma luz direcional fraca, que simula a claridade vinda das janelas.
- **Quatro luzes pontuais:** uma em cada luminária do teto, com atenuação pela distância.

Para cada luz, somamos a parcela difusa (lei de Lambert, N · L) e a especular de Blinn, que usa o vetor H, a meia distância entre a direção da luz e a do observador. A atenuação das luzes pontuais segue a fórmula clássica, com kc = 1, kl = 0,14 e kq = 0,07:

```
I = \frac{1}{k_c + k_l\,d + k_q\,d^2}\Big[\max(N\cdot L,0)\,k_d + \max(N\cdot H,0)^{\alpha}\,k_s\Big]
```

Cada peça tem um material com cor difusa, textura opcional, cor especular, expoente de brilho (α) e cor emissiva. Criamos predefinições para os tipos de superfície mais comuns da sala:

Material	Especular	Brilho (α)	Comportamento
Metal	0,80	96	Difusa escura e reflexo forte e concentrado
Plástico	0,45	32	Cor na difusa e reflexo branco de tamanho médio
Madeira	0,06	12	Quase só difusa, reflexo mínimo e espalhado
Fosco	0,01	4	Espalha a luz por igual, sem reflexo visível (lona, quadro)

O material é herdado pela árvore da cena: basta defini-lo na raiz do objeto, e só as peças diferentes o sobrescrevem. Com isso, a mesa do professor recebe madeira escura no corpo e metal escovado apenas nos puxadores, por exemplo.

Mapeamento de textura

Aplicamos texturas em cinco superfícies da sala e usamos outras três imagens como slides do projetor. Todas as imagens foram geradas por scripts próprios, o que evitou problemas de licença e nos deu texturas que se repetem sem emendas visíveis.

- **Carregamento:** as imagens são lidas com stb_image e enviadas à GPU. Como a imagem vem de cima para baixo e o OpenGL espera a primeira linha embaixo, ativamos `stbi_set_flip_vertically_on_load(true)`.
- **Filtragem e mipmaps:** geramos mipmaps e usamos filtragem trilinear (`GL_LINEAR_MIPMAP_LINEAR`). Sem isso, superfícies vistas de longe ou em ângulo, como o tampo das carteiras no fundo da sala, cintilariam.
- **Repetição (tiling):** com `GL_REPEAT`, UVs acima de 1 repetem a imagem. Cada material tem um fator de escala da UV, e as primitivas já geram UVs em metros, então o veio da madeira mantém o mesmo tamanho em qualquer peça.
- **Cor do material × textura:** no shader, a cor da textura é multiplicada pela cor difusa do material. Assim uma mesma imagem pode ser tingida em tons diferentes.

Superfície	Textura	Repetição
Tampo da carteira, assento e encosto da cadeira	Laminado madeirado claro	A cada 0,5 m
Mesa do professor (tampo, painéis e gavetas)	Madeira escura com veio	A cada 0,6 m
Corpo do projetor	Plástico cinza texturizado	A cada 25 cm
Lona da tela retrátil	Trama de tecido	A cada 25 cm
Lixeira	Polipropileno azul	A cada 25 cm

As peças em que a textura não acrescentaria realismo, como as pás do ventilador em movimento e a superfície do quadro, ficaram apenas com cor e brilho do material.

Interações

Implementamos quatro interações, uma a mais que o mínimo exigido. Todas as animações avançam pelo tempo real entre quadros (deltaTime), para terem a mesma duração em qualquer máquina.

Tecla	Interação	Técnica principal
`T`	Abrir e recolher a tela retrátil	Escala em Y com pivô no topo
`V` e `Shift` + `V`	Ligar o ventilador e trocar a velocidade	Rotação hierárquica do rotor
`P`, `←` e `→`	Ligar o projetor e trocar o slide	Textura projetiva
`L`	Acender e apagar as luzes da sala	Alteração das luzes e da emissiva

Tela retrátil. A lona leva cerca de 2,5 s para abrir ou recolher. Em vez de uma velocidade constante, passamos o progresso por uma função smoothstep, que faz a tela acelerar no começo e frear no fim, como uma tela real. Se a tecla for apertada no meio do curso, só o sentido do progresso muda, então o movimento se inverte sem saltos. A barra inferior acompanha a lona por translação.

Ventilador. Ao ligar, o rotor acelera até a velocidade escolhida; ao desligar, desacelera até parar, imitando a inércia de um motor. A cada quadro somamos ao ângulo a velocidade angular vezes o deltaTime. Limitamos as velocidades a 2, 3 ou 4 voltas por segundo para evitar o efeito “roda de carroça”, em que pás rápidas demais parecem paradas ou girando ao contrário na tela.

Projetor. Esta foi a técnica mais elaborada do projeto. Em vez de simplesmente trocar a textura da lona, tratamos o projetor como uma segunda câmera, com matriz view (a partir da lente) e matriz de projeção próprias. No fragment shader, cada ponto da cena é levado ao espaço dessa câmera e a posição na imagem vira a coordenada UV do slide:

```
uv = \frac{1}{2}\left(\frac{(P_{\text{proj}}\,V_{\text{proj}}\,p)_{xy}}
{w}\right) + \frac{1}{2}
```

O slide funciona como uma luz que carrega uma imagem e cai onde o feixe bater. Com a tela aberta, ele aparece na lona; recolhida, no quadro e na parede. O ângulo de abertura foi calculado a partir da distância entre a lente e a tela (cerca de 2,9 m), como no throw ratio de um projetor real. Como o projetor fica acima da tela e inclinado para baixo, a imagem sai num trapézio levemente mais largo embaixo (efeito keystone), o que deixou a cena mais realista. A intensidade também cai com o quadrado da distância, e a lente acende com uma cor emissiva.

Luz da sala. Apertar `L` desliga as quatro luzes pontuais, apaga o brilho emissivo dos tubos, reduz a luz ambiente e bascula a tecla do interruptor na parede. A luz de preenchimento continua ativa, então a sala fica escura, mas não preta, como uma sala de dia com as lâmpadas apagadas.

Considerações finais

O projeto cumpriu todos os requisitos da disciplina: nove objetos modelados no código, as três transformações, navegação com W/A/S/D e mouse, quatro interações, iluminação com materiais distintos e mapeamento de textura. Dividimos o trabalho de modo que cada membro modelasse três objetos e ficasse responsável por uma área do código (navegação e montagem, iluminação e materiais, interações), e a organização em camadas permitiu juntar tudo com poucos conflitos.

Reconhecemos duas limitações. A cena não tem sombras: como não implementamos shadow mapping, uma pessoa à frente do projetor recebe a imagem no corpo, mas não projeta sombra na tela. A câmera também não tem colisão, então é possível atravessar paredes e móveis. Ambas seriam os próximos passos naturais para aumentar o realismo.